

SETUP

git clone <url> — download a repo from GitHub to your computer

git remote add upstream <url> — link the instructor's original repo

git checkout -b <name> — create a new branch and switch to it

git status — show current branch and changed files

LOOK INSIDE .git/

cat .git/HEAD — see where you are now (branch name, or SHA if detached)

cat .git/refs/heads/main — a branch is just this one file with one SHA

git rev-parse HEAD — print the full SHA of your current commit

git cat-file -p HEAD — show what is inside a commit (tree, parent, author, message)

COMMITTS

git add <file> — stage a file (put it in the box for the next commit)

git commit -m "feat: ..." — record a snapshot with a message

git commit -am "..." — stage all TRACKED files + commit in one step

git log --oneline — history, one line per commit

git log --oneline --graph — history drawn as a branch diagram

Message format: **type: description** (feat, fix, docs, style, refactor, chore)

CLEAN HISTORY (rebase -i)

git rebase -i HEAD~3 — open the plan editor for the last 3 commits (oldest on top!)

pick — keep the commit as-is · **reword** — keep it, retype the message

squash — melt into the commit above (messages combine)

fixup — melt into the commit above, throw this message away

drop — delete the commit completely (careful!)

GOLDEN RULE: never rebase commits already pushed to a shared branch

VIM SURVIVAL

i — start typing (INSERT) · **Esc** — stop typing · **ciw** — replace one word

dd — delete a line · **:wq** — save & quit · **:q!** — quit WITHOUT saving (safe cancel)

MERGE CONFLICT

git merge <branch> — combine that branch into this one

Conflict markers: <<<<<< **yours** ===== **theirs** >>>>>>

Fix: edit the file, delete ALL markers, keep what you want

git add <file> && git commit — finish the merge after fixing

git merge --abort — panic button: cancel the merge, start over

CHERRY-PICK & STASH

git cherry-pick <sha> — copy ONE commit from another branch to here

Rule: only works if this branch does NOT already have that change

git cherry-pick --abort — cancel a stuck cherry-pick

git stash push -m "name" — shelve uncommitted changes, with a label

git stash -u — also shelve brand-new (untracked) files

git stash list — show the shelf · **git stash pop** — take back & remove from shelf

HOOKS

cp hooks-examples/pre-commit .git/hooks/ — install a hook

chmod +x .git/hooks/pre-commit — make it executable (required!)

pre-commit runs before each commit; exit 1 = commit blocked, exit 0 = allowed

RESCUE (reflog)

git reflog — diary of everywhere HEAD has been (nothing is lost!)

git reset --hard HEAD~2 — DANGER: move back 2 commits, wipe changes

git reset --hard HEAD@{N} — time-travel back to a reflog entry

git checkout -b rescue — save yourself from detached HEAD

git restore --staged <file> — un-stage a file (undo git add)

SUBMIT

git push -u origin <branch> — upload your branch to GitHub

Then on GitHub: **Compare & pull request** → base: instructor's main

Exercise 1: send your repo URL · Exercise 2: PR titled [workshop] Your Name

じゅんぴ

git clone <url> — GitHub の リポジトリを パソコンに ダウンロードする

git remote add upstream <url> — せんせいの もとの リポジトリをつなぐ

git checkout -b <name> — あたらしい ブランチをつくってうつる

git status — いまの ブランチと へんこうした ファイルを みる

.git/ の なかを みる

cat .git/HEAD — いま どこに いるか みる (ブランチめい、detached なら SHA)

cat .git/refs/heads/main — ブランチは SHA が ひとつ はいった ファイル

git rev-parse HEAD — いまの コミットの SHA を ぜんぶ みる

git cat-file -p HEAD — コミットの なかみを みる (tree ・ おや ・ さくしゃ ・ メッセージ)

コミット

git add <file> — ファイルを ステージする (つぎの コミットの はここに いれる)

git commit -m "feat: ..." — メッセージつきで スナップショットを きろくする

git commit -am "..." — ついせきちゅうの ファイルを ぜんぶ add して コミット

git log --oneline — れきしを 1ぎょうずつ みる

git log --oneline --graph — れきしを ブランチの ずで みる
メッセージの かたち: **type: せつめい** (feat, fix, docs, style, refactor, chore)

れきしを きれいに (rebase -i)

git rebase -i HEAD~3 — さいきん 3つの コミットの けいかくを ひらく (いちばん うえが いちばん ふるい！)

pick — そのまま つかう ・ **reword** — メッセージだけ かきなおす

squash — うえの コミットに まぜる (メッセージも まぜる)

fixup — うえの コミットに まぜて、メッセージは すてる

drop — コミットを ぜんぶ けす (きけん！)

ゴールデンルール: みんなで つかう ブランチに push した コミットは ぜったい rebase しない

vim サバイバル

i — かきはじめる ・ **Esc** — かくのを やめる ・ **ciw** — ことばを 1つかえる

dd — 1ぎょう けす ・ **:wq** — ほぞんして とじる ・ **:q!** — ほぞんしない で とじる (あんぜんに キャンセル)

マージ コンフリクト

git merge <branch> — その ブランチを いまの ブランチに あわせる
コンフリクトの しるし: <<<<<<< **じぶん** ===== **あいて** >>>>>>>

なおしかた: ファイルを ひらいて、しるしを ぜんぶ けして、のこしたいものを のこす

git add <file> && git commit — なおしたら マージを かんせいさせる

git merge --abort — パニックボタン: マージを やめて やりなおす

cherry-pick と stash

git cherry-pick <sha> — べつの ブランチから コミットを 1つだけ コピーする

ルール: この ブランチが まだ もっていない コミットだけ コピーできる

git cherry-pick --abort — とまった cherry-pick を キャンセルする

git stash push -m "なまえ" — コミットまえの へんこうを なま えつきで しまう

git stash -u — あたらしい (みついせきの) ファイルも いっしょに しまう

git stash list — たなを みる ・ **git stash pop** — とりだして たなから けす

フック

cp hooks-examples/pre-commit .git/hooks/ — フックを いれる

chmod +x .git/hooks/pre-commit — じっこうできるように する (ひっす！)

pre-commit は コミットの まえに うごく。exit 1 = ブロック、exit 0 = OK

ぎゅうしゅつ (reflog)

git reflog — HEAD が いた ばしょ ぜんぶの にっき (なにも きえていない！)

git reset --hard HEAD~2 — きけん: 2つ もどって へんこうも けす

git reset --hard HEAD@{N} — にっきの ばしょへ タイムトラベルで もどる

git checkout -b rescue — detached HEAD から じぶんを たすける

git restore --staged <file> — ステージを とりけす (git add の とりけし)

ていしゅつ

git push -u origin <branch> — じぶんの ブランチを GitHub に アップロードする

GitHub で: **Compare & pull request** → base は せんせいの main
えんしゅう1: リポジトリの URL を おくる ・ えんしゅう2: PR の タイトルは [workshop] なまえ